Lecture Notes / version 0.1

Machine Learning A

Casper Moes (hwd362) 31. October 2026

Indhold

1 Concentration of measure inequalities **1**

a Markow's inequality . . . 1

b Chebychew's inequality . . . 2

c Hoeffding's inequality . . . 2

2 Neural Networks **4**

a The Neuron . . . 4

a.1 Pre-Activation . . . 4

a.2 Activation and output . . . 4

a.3 Activation functions . . . 4

b Gradient descent . . . 6

c Backpropagation . . . 7

c.1 The ouput layer . . . 7

c.2 Hidden layers . . . 8

3 Principal Component Analysis **8**

a Definitions . . . 8

a.1 Dimensions . . . 8

b The objective of PCA . . . 9

c Optimal reconstruction . . . 9

1 Concentration of measure inequalities

a) Markow's inequality

Statement: For any non-negative stochastic variable X and with $\epsilon > 0$ we have

$$P(X \geq \epsilon) \leq \frac{E[X]}{\epsilon}$$

.

Proof: Define $Y := 1(X \geq \epsilon)$ then $Y \leq \frac{X}{\epsilon}$ (if $0 \leq X < \epsilon$ then $Y = 0 \leq \frac{X}{\epsilon}$ and if $X \geq \epsilon \iff \frac{X}{\epsilon} \geq 1$ then $Y = 1 \leq \frac{X}{\epsilon}$) therefore $P(X \geq \epsilon) = P(Y = 1) = E[Y] \leq E[\frac{X}{\epsilon}] = \frac{E[X]}{\epsilon}$.

This can be rewritten by defining $a = \frac{\epsilon}{E[X]}$ ($E[X] \neq 0$) and we have

$$P(X \geq aE[X]) \leq \frac{1}{a}$$

. That is, the probability of an event a times away from the stochastic variables mean value is decaying with $1/a$.

b) Chebychew's inequality

Statement:

$$P(|X - E[X]| \geq \epsilon) \leq \frac{V(X)}{\epsilon^2}$$

Proof: For any $\epsilon > 0$ we have (by Markow's inequality) $P((X - E[X])^2 \geq \epsilon^2) \leq \frac{E[(X-E[X])^2]}{\epsilon^2} = \frac{V(X)}{\epsilon^2}$. Finally we note that $|X - E[X]| \geq \epsilon \iff (X - E[X])^2 \geq \epsilon^2$ and thus $P(|X - E[X]| \geq \epsilon) \leq \frac{V(X)}{\epsilon^2}$.

c) Hoeffding's inequality

Statement: Given the independent stochastic variables $X_1, \ldots, X_n$ where there exists a_i, b_i for $i \in \{1, \ldots, n\}$ such that $X_i \in [a_i, b_i]$. For all $\epsilon > 0$,

$$P\left(\sum_{i=1}^{n} X_i - E\left(\sum_{i=1}^{n} X_i\right) \geq \epsilon\right) \leq e^{-\frac{2\epsilon^2}{\sum_{i=1}^{n}(b_i - a_i)^2}}.$$

Proof:

$$\begin{aligned} P\left(\sum_{i=1}^{n} X_i - E\left(\sum_{i=1}^{n} X_i\right) \geq \epsilon\right) &= P\left(e^{\lambda\left(\sum_{i=1}^{n} X_i - E\left(\sum_{i=1}^{n} X_i\right)\right)} \geq e^{\lambda\epsilon}\right) \\ &\leq \frac{E\left[e^{\lambda\left(\sum_{i=1}^{n} X_i - E\left(\sum_{i=1}^{n} X_i\right)\right)}\right]}{e^{\lambda\epsilon}} \end{aligned}$$

The nominator can be reduced. Firstly we use that the expectation operator is linear so that $E\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} E[X_i]$

$$E\left[e^{\lambda\left(\sum_{i=1}^{n} X_i - E\left(\sum_{i=1}^{n} X_i\right)\right)}\right] = E\left[e^{\lambda\left(\sum_{i=1}^{n} X_i - \sum_{i=1}^{n} E[X_i]\right)}\right] = E\left[e^{\lambda \sum_{i=1}^{n}(X_i - E[X_i])}\right]$$

Now using the properties of the exponential function and Hoeffdings lemma (proof will follow)

$$
\begin{aligned}
E\left[e^{\lambda \sum_{i=1}^{n} X_i - E[X_i]}\right] &= E\left[\prod_{i=1}^{n} e^{\lambda(X_i - E[X_i])}\right] \\
&= \prod_{i=1}^{n} E\left[e^{\lambda(X_i - E[X_i])}\right] \\
&\leq \prod_{i=1}^{n} e^{\lambda^2 (b_i - a_i)^2/8} \\
&= e^{\lambda^2/8 \sum_{i=1}^{n} (b_i - a_i)^2}.
\end{aligned}
$$

Taking stock: so far we have

$$
P\left(\sum_{i=1}^{n} X_i - E\left(\sum_{i=1}^{n} X_i\right) \geq \epsilon\right) \leq e^{\lambda^2/8 \sum_{i=1}^{n} (b_i - a_i)^2 - \lambda\epsilon}.
$$

However we want the tightest probability bound, and therefore we minimize the right hand side expression wrt. $\lambda > 0$.

$$
\lambda^* = \arg\min_{\lambda} \; e^{\lambda^2/8 \sum_{i=1}^{n} (b_i - a_i)^2 - \lambda\epsilon} = \arg\min_{\lambda} \left\{ \frac{\lambda^2}{8} \sum_{i=1}^{n} (b_i - a_i)^2 - \lambda\epsilon \right\},
$$

with necessary first order condition

$$
0 = \frac{\lambda^*}{4} \sum_{i=1}^{n} (b_i - a_i)^2 - \epsilon \iff \lambda^* = \frac{4\epsilon}{\sum_{i=1}^{n} (b_i - a_i)^2}.
$$

Substituting this into the stock-taking expression yields the theorem statement

$$
\begin{aligned}
e^{(\lambda^*)^2/8 \sum_{i=1}^{n} (b_i - a_i)^2 - \lambda^* \epsilon} &= e^{\left(\frac{4\epsilon}{\sum_{i=1}^{n} (b_i - a_i)^2} \cdot\right)^2 / 8 \sum_{i=1}^{n} (b_i - a_i)^2 - \frac{4\epsilon^2}{\sum_{i=1}^{n} (b_i - a_i)^2}} \\
&= e^{\frac{2\epsilon^2}{\sum_{i=1}^{n} (b_i - a_i)^2} - \frac{4\epsilon^2}{\sum_{i=1}^{n} (b_i - a_i)^2}} \\
&= e^{-\frac{2\epsilon^2}{\sum_{i=1}^{n} (b_i - a_i)^2}}.
\end{aligned}
$$

Taken all together we have obtained Hoeffding Inequality:

$$
P\left(\sum_{i=1}^{n} X_i - E\left(\sum_{i=1}^{n} X_i\right) \geq \epsilon\right) \leq e^{-\frac{2\epsilon^2}{\sum_{i=1}^{n} (b_i - a_i)^2}}.
$$

2 Neural Networks

a) The Neuron

a.1 Pre-Activation

The preactivation of neuron i is given by

$$\begin{aligned} a_i(\mathbf{x}) &= \sum_{j=1}^{d} w_{ij}x_j + w_{i0} \\ &= \mathbf{w}_i'\mathbf{x}_{+1}, \end{aligned}$$

where $\mathbf{x}_{+1} \in \mathbb{R}^{d+1}$ denotes the argmentet vector

$$\mathbf{x}_{+1} = \begin{bmatrix} 1 & x_1 & \cdots & x_d \end{bmatrix}.$$

a.2 Activation and output

Applying a (non-linear) activation function $h_i : \mathbb{R} \to \mathbb{R}$ yields the output of neuron i

$$z_i(\mathbf{x}) = h_i(a_i(\mathbf{x})) = h_i\left(\sum_{j=1}^{d} w_{ij}x_j + w_{i0}\right).$$

That is each neuron z_i has its own weights (and bias) $\mathbf{w} \in \mathbb{R}^{d+1}$ and activation function h_i. The latter is usually the same for all neurons except the output-neurons (in the top layer).

a.3 Activation functions

Linear activation function: For the network structure to have any reason to exist the activation functions h_i must be non-linear. If not the neural network $f : \mathbb{R}^d \to \mathbb{R}^K$ will simply be a linear function of the inputs aswell.

To see this note that each neuron in the inputlayer would output the following functional form

$$\begin{aligned} h\left(\sum_{j=1}^{d} w_{ij}x_j + w_{i0}\right) &= \sum_{j=1}^{d} \underbrace{aw_{ij}}_{\tilde{w}_{ij}} x_j + \underbrace{aw_{i0} + b}_{\tilde{w}_{i0}} \\ &= \sum_{j=1}^{d} \tilde{w}_{ij}x_j + \tilde{w}_{i0}, \end{aligned}$$

and since each pre-activation is simply a affine linear function of the previous affine linear

functions all later neurons will output something of the form:

$$
\begin{aligned}
h\left(a\left(\begin{bmatrix} z_1 & z_2 & \cdots & z_K \end{bmatrix}\right)\right) &= h\left(\sum_{k=1}^{K} w_k z_k + w_0\right) \\
&= \sum_{k=1}^{K} \hat{w}_k z_k + \hat{w}_0 \\
&= \sum_{k=1}^{K} \hat{w}_k \left(\sum_{j=1}^{d} \tilde{w}_{kj} x_j + \tilde{w}_{k0}\right) + \hat{w}_0 \\
&= \sum_{k=1}^{K} \hat{w}_k \sum_{j=1}^{d} \tilde{w}_{kj} x_j + \underbrace{\sum_{k=1}^{K} \hat{w}_k \tilde{w}_{k0} + \hat{w}_0}_{\bar{w}_0} \\
&= \sum_{k=1}^{K} \sum_{j=1}^{d} \hat{w}_k \tilde{w}_{kj} x_j + \bar{w}_0 \\
&= \sum_{j=1}^{d} \sum_{k=1}^{K} \hat{w}_k \tilde{w}_{kj} x_j + \bar{w}_0 \\
&= \sum_{j=1}^{d} x_j \underbrace{\sum_{k=1}^{K} \left(\hat{w}_k \tilde{w}_{kj}\right)}_{\bar{w}_j} + \bar{w}_0 \\
&= \sum_{j=1}^{d} \bar{w}_j x_j + \bar{w}_0.
\end{aligned}
$$

This then also goes for all other layers recieving these outputs (by the exact same argument as above). So in summary: if h_i is linear for all i then there is no purpose to the neural network structure and the simple affine linear model of linear regression fits the model just as well.

Well known activation functions (greatest hits). Here i list some of the most common activation functions. Some are more common for outputlayers for example logistic activation and are rarely used in hidden layers.

- Step / Threshold
$$h(a) = \mathbb{I}(a > 0),$$
with range $\mathcal{R} \in \{0, 1\}$
- Hyperbolic tangent
$$h(a) = tanh(a),$$
with range $\mathcal{R} \in (-1, 1)$ and with a visual form much like *alternative sigmoid.*
- Rectified linear unit (ReLU)
$$h(a) = \max(0, a),$$
with range $\mathcal{R}[0, \infty)$. Note that there the derivative of the loss function wrt. pre-

activation weights is uninformative in the regrion $a < 0$. By that i mean $\partial h_i / \partial w_{ij} = 0$ if $a_i(\mathbf{x}, \mathbf{w}) < 0$ and so it does not help us adjust weights to decrease loss in these negative regions.

- Fermi / logistic
$$h(a) = \frac{1}{1 + \exp\{-a\}},$$
with range $\mathcal{R} \in (0, 1)$.
- Alternative sigmoid
$$h(a) = \frac{a}{1 + |a|},$$
with range $\mathcal{R}$
- Exponential LU (ELU, $\alpha > 0$)
$$h(a) = \begin{cases} a & a \geq 0 \\ \alpha(\exp\{a\} - 1) & a < 0 \end{cases}$$
with range $\mathcal{R} \in (-\alpha, \infty)$.

b) Gradient descent

To optimize a neural network with error function $E = \sum_{n=1}^{N} E_n$ (as with loglikelihood functions) we introduce gradient descent. The bare form of this algorithm is simply

$$\begin{aligned} \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} + \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t)} \\ &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}}, \end{aligned}$$

with learning rate $\eta > 0$ determining how large steps are in proportion to the gradient.

Often we add a *momentum term* (in an attempt to improve performance) which intuitively works like a heavy ball. When a heavy ball has gained momentum from accelerating down a steep hill it will roll over local minimums and further down towards the global minimum (hopefully). Formally we have

$$\begin{aligned} \mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}} + \mu \boldsymbol{\Delta}_{\mathbf{w}}{}^{(t-1)} \\ &= \mathbf{w}^{(t)} - \eta \nabla E|_{\mathbf{w}^{(t)}} + \mu \left(\mathbf{w}^{(t)} - \mathbf{w}^{(t-1)} \right), \end{aligned}$$

with momentum parameter $\mu > 0$.

The question now really is how to calculate the elements of $\nabla E|_{\mathbf{w}^{(t)}}$ given by

$$\frac{\partial E}{\partial w_{ij}} = \sum_{n=1}^{N} \frac{\partial E_n}{\partial w_{ij}},$$

for $(j,i) \in \mathcal{A}$. Recall that $\mathcal{A}$ contains all the edges connection the output of neuron j to the input of neuron i which is scaled by w_{ij}. This is answered in the next section.

c) Backpropagation

The goal is to calculate $\frac{\partial E_n}{\partial w_{ij}}$ for every $(j,i) \in \mathcal{A}$.[1] Applying the chain rule to every element of the error function $E = \sum_{n=1}^{N} E_n$ yields the following. Note that E_n is a function of z_i for all i in the network. and that any z_i is a function of z_j where $(j,i) \in \mathcal{A}$. Therefore the chain rule yields

$$\begin{aligned}\frac{\partial E_n}{\partial w_{ij}} &= \frac{\partial E_n}{\partial z_i}\frac{\partial z_i}{\partial a_i}\frac{\partial a_i}{\partial w_{ij}} \\ &= \frac{\partial E_n}{\partial z_i} h_i'(a_i)\,[\mathbf{z}_n]_j\end{aligned}$$

where the last step uses that

$$z_i(\mathbf{z}) \equiv h_i(a_i(\mathbf{z})) \implies \frac{\partial z_i}{\partial a_i} = h_i'(a_i),$$

where $\mathbf{z} \in \mathbb{R}^M$ contains all the inputs from the previous layer to neuron i. For the last factor we used that

$$a_i(\mathbf{z}_n) = w_{i0} + \sum_{m=1}^{M} w_{im}\,[\mathbf{z}_n]_m \implies \frac{\partial a_i}{\partial w_{ij}} = [\mathbf{z}_n]_j\,.$$

Therefore the missing factor to be calculated is $\frac{\partial E_n}{\partial z_i}$ which we will now turn to.

c.1 The ouput layer

For the output layer $\frac{\partial E_n}{\partial z_i}$ can be directly calculated. To see this let the loss function be the sum of squared errors

$$\begin{aligned}E = \frac{1}{2}\sum_{n=1}^{N} ||f(\mathbf{x}_n;\mathbf{w}) - \mathbf{y}_n||^2 &= \frac{1}{2}\sum_{n=1}^{N}\left(\sqrt{\sum_{k=1}^{K}\left([f(\mathbf{x}_n;\mathbf{w})]_k - [\mathbf{y}_n]_k\right)^2}\right)^2 \\ &= \frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K}\left([f(\mathbf{x}_n;\mathbf{w})]_k - [\mathbf{y}_n]_k\right)^2 \\ &= \frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K}\left([\mathbf{z}_n]_k - [\mathbf{y}_n]_k\right)^2\end{aligned}$$

[1] Why not for $(j,i) \neq \mathcal{A}$? These weights simply do not exist. The model only has weights symbolizing the strength of the connect from neuron j to neuron i. There is no connection from i back to j so this w_{ji} does not exist.

for $k \in V_{output}$. Therefore we have (for $i \in V_{output}$ and $(j,i) \in \mathcal{A}$)

$$\begin{aligned}\frac{\partial E_n}{\partial z_i} &= \frac{\partial}{\partial z_i}\frac{1}{2}\sum_{k=1}^{K}\left([\mathbf{z}_n]_k - [\mathbf{y}_n]_k\right)^2 \\ &= [\mathbf{z}_n]_i - [\mathbf{y}_n]_i\end{aligned}$$

For $h_i(a_i) = a_i \implies h_i'(a_i) = 1$ where $i \in V_{output}$ we thus have

$$\begin{aligned}\frac{\partial E}{\partial w_{ij}} &= \sum_{n=1}^{N}\frac{\partial E_n}{\partial z_i}h_i'(a_i)z_j \\ &= \sum_{n=1}^{N}\frac{\partial E_n}{\partial z_i}\left[\mathbf{z}_n\right]_j \\ &= \sum_{n=1}^{N}\left([\mathbf{z}_n]_i - [\mathbf{y}_n]_i\right)\left[\mathbf{z}_n\right]_j\end{aligned}$$

c.2 Hidden layers

For hidden layers we apply the apply the chainrule to obtain

$$\begin{aligned}\frac{\partial E}{\partial z_j} &= \sum_{k \text{ in } (j,k)\in\mathcal{A}}\frac{\partial E}{\partial z_k}\frac{\partial z_k}{\partial z_j} \\ &= \sum_{k \text{ in } (j,k)\in\mathcal{A}}\frac{\partial E}{\partial z_k}\frac{\partial z_k}{\partial a_k}\frac{\partial a_k}{\partial z_j} \\ &= \sum_{k \text{ in } (j,k)\in\mathcal{A}}\frac{\partial E}{\partial z_k}h_k'(a_k)w_{kj}.\end{aligned}$$

The partial derivatives of the layer just before V_{ouput} is then calculated by pluging in $\frac{\partial E}{\partial z_k}$, $k \in V_{output}$ and $(j,k) \in \mathcal{A}$. Then the third layers uses the second layers calculated $\frac{\partial E}{\partial z_j}$ and so on. This is sometimes referred to as the backwards pass.

3 Principal Component Analysis

a) Definitions

a.1 Dimensions

- D is the dimensionality of the original dataset X (sampled from $\mathcal{X}$). As an example consider the face database where each picture is 92×122 pixels. Then the dimension D of the input space is $D = 10304$.
- $d < D$ is the dimensionality of the outputspace of PCA i.e. the dimensionality we have reduced the dataset to using PCA.
- N is the number of observations in X

- X can then be defined by stacking all N observations $\mathbf{x}_i' \in \mathbb{R}^D$ as the rows to obtain $X \in \mathbb{R}^{N \times D}$. This is very much like the design matrix in linear regression).
- $U_d \in \mathbb{R}^{D \times d}$ takes in d dimensional enocded data and outputs D dimensional reconstructed data (where we add $\mathbf{b} \in \mathbb{R}^D$ for the final output $f_{dec}(\mathbf{z}_i) = \mathbf{b} + U_d \mathbf{z}_i$).

b) The objective of PCA

The goal of PCA is to take a dataset X sampled from $\mathcal{X}$ with datapoints $\mathbf{x}_i \in \mathbb{R}^D$, $i = 1, \ldots,$ and linearly map them into $\mathbf{z}_i \in \mathbb{R}^d$ where $d < D$ while minimizing the reconstruction error

$$J = \frac{1}{N} \sum_{n=1}^{N} ||\mathbf{x}_n - f_{dec}(\mathbf{z}_n)||_2^2,$$

where

$$f_{dec}(\mathbf{z}_n) = \mathbf{b} + U_m \mathbf{z}_n,$$

where $\mathbf{z}_i \in \mathbb{R}^d$, $\mathbf{b} \in \mathbb{R}^D$ and $U_d \in \mathbb{R}^{D \times d}$ is the *decoding* function.

In other words we are looking for the *encoding* $f_{enc} : \mathbb{R}^D \to \mathbb{R}^d$, $d < D$

$$\min_{\mathbf{b}, U_d \{\mathbf{z}_1, \ldots, \mathbf{z}_N\}} \frac{1}{N} \sum_{n=1}^{N} ||\mathbf{x}_n - f_{dec}(\mathbf{z}_n)||_2^2$$

c) Optimal reconstruction

Let U (with the columns $\mathbf{u}_1, \ldots, \mathbf{u}_D \in \mathbb{R}^D$) be an orthonomal basis (spanning the the whole $\mathbb{R}^D$ with unit lenght basisvectors)

$$\mathbf{u}_k' \mathbf{u}_j = \begin{cases} 1 & k = j, \\ 0 & k \neq j. \end{cases}$$

Then every vector / observation $\mathbf{x}_i \in \mathbb{R}^D$ can be represented as a linear combination of the basisvectors in U

$$\mathbf{x}_i = \sum_{j=1}^{D} a_j \mathbf{u}_j. \tag{3.1}$$

We now want to determine a_j. Premultiply (3.1) by any of the new basisvectors $\mathbf{u}_k'$, $k \in \{1, \ldots, D\}$

$$\begin{aligned} \mathbf{u}_k' \mathbf{x}_i &= \mathbf{u}_k' \sum_{j=1}^{n} a_j \mathbf{u}_j = \sum_{j=1}^{n} \mathbf{u}_k' a_j \mathbf{u}_j = \sum_{j=1}^{n} a_j \mathbf{u}_k' \mathbf{u}_j \\ &= \underbrace{a_1 \mathbf{u}_k' \mathbf{u}_1 + \cdots +}_{=0} a_k \mathbf{u}_k' \mathbf{u}_k \underbrace{+ \cdots + a_\ell \mathbf{u}_k' \mathbf{u}_\ell}_{=0} \\ &= a_k, \end{aligned}$$

where we have used the pairwise orthogonality of the basis vectors and the unit length of any $\mathbf{u}_k$ such that $\mathbf{u}_k'\mathbf{u}_k = ||\mathbf{u}_k||_2^2 = 1^2$. Inserting $a_k = \mathbf{u}_k'\mathbf{x}_i$ for all $k \in \{1, \ldots, D\}$ into (3.1) yields

$$\mathbf{x}_i = \sum_{j=1}^{d} \left(\mathbf{u}_j'\mathbf{x}_i\right) \mathbf{u}_j.$$

Note that this only works because of the orthonormal restriction on the basis U.